

Tuyển tập Bài tập Thực hành: Ngăn xếp (Stack) & Ngăn xếp Đơn điệu (Monotone Stack)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng tổng hợp Danh mục 16 Bài tập Phân cấp

STT	Mã bài (Click mở đề)	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
1	CSES-1645	Nearest Smaller Values	CSES	★ Cơ bản	Previous Smaller Element (PSE)	$O(N)$
2	CF-5C	Longest Regular Bracket Sequence	Codeforces	★ Cơ bản	Stack kiểm tra dãy ngoặc & DP	$O(N)$
3	LC-20	Valid Parentheses	LeetCode	★ Cơ bản	Ngăn xếp kiểm tra ngoặc đa loại	$O(N)$
4	LC-496	Next Greater Element I	LeetCode	★ Cơ bản	Next Greater Element (NGE)	$O(N + M)$
5	CF-1598C	Delete Two Elements	Codeforces	★ Cơ bản	Xử lý cặp giá trị trung bình	$O(N \log N)$
6	CSES-1142	Advertisement	CSES	★★ Trung cấp	Largest Rectangle in Histogram	$O(N)$
7	CSES-1147	Maximum Building I	CSES	★★ Trung cấp	Maximal Rectangle trong ma trận 2D	$O(N \cdot M)$
8	ABC-189C	Mandarin Orange	AtCoder	★★ Trung cấp	Biến thể Histogram	$O(N)$
9	CF-1313C2	Skyscrapers (hard version)	Codeforces	★★ Trung cấp	Monotone Stack kết hợp DP đơn đỉnh	$O(N)$

STT	Mã bài (Click mở đề)	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
10	CF-280B	Maximum Xor Secondary	Codeforces	☆☆ Trung cấp	XOR 2 số lớn nhất bằng Monotone Stack	$O(N)$
11	LC-739	Daily Temperatures	LeetCode	☆☆ Trung cấp	Khoảng cách tới ngày ấm hơn (NGE)	$O(N)$
12	LC-907	Sum of Subarray Minimums	LeetCode	☆☆☆ Nâng cao	Tính đóng góp mảng con, khử đếm trùng	$O(N)$
13	CF-1407D	Discrete Centrifugal Jumps	Codeforces	☆☆☆ Nâng cao	DP bước nhảy kết hợp 2 Monotone Stack	$O(N)$
14	CF-1691D	Max GEQ Sum	Codeforces	☆☆☆ Nâng cao	So sánh cực đại với tổng mảng con	$O(N \log N)$
15	CSES- 1148	Maximum Building II	CSES	☆☆☆ Nâng cao	Đếm số HCN mọi kích thước ma trận	$O(N \cdot M)$
16	CF-1195E	OpenStreetMap	Codeforces	☆☆☆ Nâng cao	Cửa sổ trượt 2D Monotonic Deque	$O(N \cdot M)$

● MỨC ĐỘ 1: KHỞI ĐỘNG & STACK CƠ BẢN (WARM-UP)

1. CSES 1645 - Nearest Smaller Values

- **Link đề bài:** [CSES 1645](#)
- **Tóm tắt:** Cho dãy N số nguyên ($N \leq 2 \cdot 10^5$). Với mỗi phần tử $A[i]$, tìm vị trí $j < i$ gần nhất có $A[j] < A[i]$. Nếu không có, in ra 0.
- **Ý tưởng:** Duy trì Monotone Increasing Stack lưu chỉ số. Với mỗi phần tử $A[i]$, liên tục `pop` các chỉ số có giá trị $\geq A[i]$. Nếu stack rỗng in ra 0, ngược lại in ra `st.top()`. Sau đó `st.push(i)`.
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

2. Codeforces 5C - Longest Regular Bracket Sequence

- **Link đề bài:** [Codeforces 5C](#)
- **Tóm tắt:** Tìm độ dài của chuỗi ngoặc đúng dài nhất và số lượng chuỗi con liên tiếp đạt độ dài cực đại đó.

- **Ý tưởng:**
- Dùng ngăn xếp lưu chỉ số của các ký tự. Khởi tạo đẩy chỉ số lính canh `-1` vào stack.
- Khi gặp `(` : Đẩy chỉ số vào stack.
- Khi gặp `)` : `st.pop()` . Nếu stack rỗng, đẩy chỉ số hiện tại vào làm lính canh mới. Nếu không rỗng, độ dài chuỗi ngoặc đúng kết thúc tại vị trí hiện tại là `i - st.top()` . Cập nhật độ dài lớn nhất và đếm tần suất.
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

3. LeetCode 20 - Valid Parentheses

- **Link đề bài:** [LeetCode 20](#)
- **Tóm tắt:** Kiểm tra tính hợp lệ của chuỗi ngoặc gồm các loại `()`, `[]`, `{}`.
- **Ý tưởng:** Duyệt từng ký tự, gặp ngoặc mở thì đẩy vào stack, gặp ngoặc đóng thì kiểm tra xem đỉnh stack có phải ngoặc mở tương ứng hay không.
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

4. LeetCode 496 - Next Greater Element I

- **Link đề bài:** [LeetCode 496](#)
- **Tóm tắt:** Với mỗi phần tử trong mảng con, tìm phần tử đầu tiên lớn hơn nó xuất hiện ở bên phải trong mảng gốc.
- **Ý tưởng:** Duyệt mảng gốc từ phải sang trái, duy trì Monotone Decreasing Stack để tìm NGE cho tất cả các phần tử trong $O(N)$. Lưu kết quả vào bảng băm `unordered_map` để trả lời truy vấn trong $O(1)$.
- **Độ phức tạp:** Thời gian: $O(N + M)$, Bộ nhớ: $O(N)$.

5. Codeforces 1598C - Delete Two Elements

- **Link đề bài:** [Codeforces 1598C](#)
- **Tóm tắt:** Đếm số cặp (i, j) sao cho khi xóa $A[i]$ và $A[j]$, giá trị trung bình cộng của mảng không đổi.
- **Ý tưởng:** Gọi trung bình cộng ban đầu là $k = \frac{2 \cdot \text{Sum}}{N}$. Bài toán quy về đếm số cặp có tổng $A[i] + A[j] = k$. Sắp xếp mảng hoặc dùng Map tần suất trong $O(N \log N)$.
- **Độ phức tạp:** Thời gian: $O(N \log N)$, Bộ nhớ: $O(N)$.

● MỨC ĐỘ 2: TRUNG CẤP (HISTOGRAM & MA TRẬN 2D)

6. CSES 1142 - Advertisement (Largest Rectangle in Histogram)

- **Link đề bài:** [CSES 1142](#)
- **Tóm tắt:** Cho biểu đồ gồm N cột liên nhau ($N \leq 2 \cdot 10^5$), cột thứ i có chiều cao h_i . Tìm diện tích hình chữ nhật lớn nhất nằm trong biểu đồ.
- **Ý tưởng:** Với mỗi cột i , tìm cột nhỏ hơn đầu tiên bên trái ($PSE[i]$) và bên phải ($NSE[i]$) bằng Monotone Increasing Stack. Diện tích lớn nhất nếu chọn cột i làm chiều cao là $h_i \cdot (NSE[i] -$

PSE[i] - 1). Lưu ý dùng `long long` cho diện tích.

- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

7. CSES 1147 - Maximum Building I (Maximal Rectangle 2D)

- **Link đề bài:** [CSES 1147](#)
- **Tóm tắt:** Cho bản đồ $N \times M$ gồm các ô đất trống (.) và cây cối (*). Tìm diện tích khu đất hình chữ nhật lớn nhất chỉ gồm toàn ô đất trống.
- **Ý tưởng:** Dồn mảng theo từng hàng: Tại hàng r , chiều cao liên tiếp các ô trống tính ngược lên trên tạo thành một biểu đồ Histogram 1D độ dài M . Áp dụng thuật toán Histogram trên mỗi hàng trong $O(M)$.
- **Độ phức tạp:** Thời gian: $O(N \cdot M)$, Bộ nhớ: $O(M)$.

8. AtCoder abc189_c - Mandarin Orange

- **Link đề bài:** [AtCoder abc189_c](#)
- **Tóm tắt:** Tìm $\max_{l \leq r} (r - l + 1) \cdot \min(A[l \dots r])$.
- **Ý tưởng:** Đây chính xác là bài toán tìm diện tích lớn nhất trong biểu đồ cột. Áp dụng Monotone Stack giải quyết trong $O(N)$ (vượt trội so với $O(N^2)$).
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

9. Codeforces 1313C2 - Skyscrapers (hard version)

- **Link đề bài:** [Codeforces 1313C2](#)
- **Tóm tắt:** Xây dựng dãy nhà sao cho chiều cao không vượt quá giới hạn cho trước m_i và tạo thành dãy đơn đỉnh (tăng dần rồi giảm dần). Tìm cấu hình có tổng chiều cao lớn nhất.
- **Ý tưởng:**
- Gọi $L[i]$ là tổng chiều cao lớn nhất nếu dãy tăng dần kết thúc tại i .
- Tìm phần tử nhỏ hơn gần nhất bên trái PSE[i] = j . Khi đó từ $j + 1$ đến i , mọi tòa nhà đều bị giới hạn bởi m_i :

$$L[i] = L[j] + m_i \cdot (i - j)$$

- Tính tương tự cho $R[i]$ theo chiều ngược lại. Đỉnh tối ưu là vị trí có $L[i] + R[i] - m_i$ lớn nhất.
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

10. Codeforces 280B - Maximum Xor Secondary

- **Link đề bài:** [Codeforces 280B](#)
- **Tóm tắt:** Với mỗi đoạn con liên tiếp độ dài ≥ 2 , lấy XOR của hai phần tử lớn nhất. Tìm giá trị XOR lớn nhất có thể.
- **Ý tưởng:**
- Cặp hai phần tử lớn nhất của một đoạn con luôn có thể thu hẹp về dạng: Một phần tử x và phần tử lớn hơn gần nhất của nó ở bên trái (PGE) hoặc bên phải (NGE).
- Dùng Monotone Decreasing Stack tìm PGE và NGE của mọi phần tử, lấy max XOR của từng cặp trong $O(N)$.
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

11. LeetCode 739 - Daily Temperatures

- **Link đề bài:** [LeetCode 739](#)
- **Tóm tắt:** Cho mảng nhiệt độ hàng ngày. Với mỗi ngày, tìm số ngày phải chờ đợi cho đến khi có một ngày ấm hơn.
- **Ý tưởng:** Ứng dụng trực tiếp của Next Greater Element (NGE). Khoảng cách chờ đợi là $\text{NGE}[i] - i$ (hoặc 0 nếu không có).
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

MỨC ĐỘ 3: NÂNG CAO & KỸ THUẬT TÍNH ĐÓNG GÓP (ADVANCED CONTRIBUTIONS)

12. LeetCode 907 - Sum of Subarray Minimums

- **Link đề bài:** [LeetCode 907](#)
- **Tóm tắt:** Tính tổng phần tử nhỏ nhất của tất cả các mảng con liên tiếp modulo $10^9 + 7$.
- **Ý tưởng:** Áp dụng kỹ thuật tính đóng góp của từng phần tử $A[i] \times L_i \times R_i$. Bắt buộc dùng dấu `>` cho bên trái và `>=` cho bên phải để triệt tiêu đếm trùng lặp khi có phần tử bằng nhau.
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

13. Codeforces 1407D - Discrete Centrifugal Jumps

- **Link đề bài:** [Codeforces 1407D](#)
- **Tóm tắt:** Từ tòa nhà i có thể nhảy tới $j > i$ nếu mọi tòa nhà ở giữa đều nhỏ hơn cả 2 tòa nhà, hoặc đều lớn hơn cả 2 tòa nhà. Tìm số bước nhảy ít nhất từ 1 đến N .
- **Ý tưởng:**
- Điều kiện nhảy tương đương với việc j là NGE hoặc NSE của i , hoặc i là PGE hoặc PSE của j .
- Dùng 2 Monotone Stack (một tăng, một giảm) để tìm các cạnh hợp lệ trên đồ thị, sau đó kết hợp Quy hoạch động $DP[j] = \min(DP[i] + 1)$ giải trong $O(N)$.
- **Độ phức tạp:** Thời gian: $O(N)$, Bộ nhớ: $O(N)$.

14. Codeforces 1691D - Max GEQ Sum

- **Link đề bài:** [Codeforces 1691D](#)
- **Tóm tắt:** Kiểm tra xem với mọi mảng con liên tiếp, giá trị lớn nhất trong mảng con có luôn $\geq$ tổng của mảng con đó hay không.
- **Ý tưởng:**
- Điều kiện vi phạm: Tồn tại mảng con $[l, r]$ có $\sum_{k=l}^r A[k] > \max(A[l \dots r])$.
- Với mỗi phần tử $A[i]$, dùng Monotone Stack tìm khoảng $[L_i, R_i]$ mà $A[i]$ là cực đại lớn nhất. Ta cần kiểm tra xem có đoạn con nào qua i nằm trong $[L_i, R_i]$ có tổng $> A[i]$ hay không (bằng cách lấy tổng lớn nhất mở rộng sang trái và sang phải nhờ Segment Tree / Sparse Table).
- **Độ phức tạp:** Thời gian: $O(N \log N)$, Bộ nhớ: $O(N)$.

15. CSES 1148 - Maximum Building II

- **Link đề bài:** [CSES 1148](#)
- **Tóm tắt:** Cho lưới $N \times M$. Với mỗi kích thước hình chữ nhật $H \times W$ ($1 \leq H \leq N, 1 \leq W \leq M$), đếm số vị trí có thể đặt được tòa nhà kích thước đó.
- **Ý tưởng:** Kết hợp Histogram trên từng hàng bằng Monotone Stack để tìm các hình chữ nhật cực đại, sau đó dùng kỹ thuật Mảng hiệu 2D (2D Difference Array) để cộng dồn số lượng hình chữ nhật con trong $O(N \cdot M)$.
- **Độ phức tạp:** Thời gian: $O(N \cdot M)$, Bộ nhớ: $O(N \cdot M)$.

16. Codeforces 1195E - OpenStreetMap

- **Link đề bài:** [Codeforces 1195E](#)
- **Tóm tắt:** Cho ma trận $N \times M$. Tìm tổng giá trị nhỏ nhất của tất cả các ma trận con kích thước $A \times B$.
- **Ý tưởng:**
- Chuyển bài toán 2D về 1D bằng Monotonic Queue (Sliding Window Minimum):
- Bước 1: Với mỗi hàng, dùng Monotonic Queue tìm giá trị nhỏ nhất trong cửa sổ độ dài $B \implies$ Thu được ma trận $N \times (M - B + 1)$.
- Bước 2: Với mỗi cột của ma trận mới, dùng Monotonic Queue tìm giá trị nhỏ nhất trong cửa sổ độ dài A .
- **Độ phức tạp:** Thời gian: $O(N \cdot M)$, Bộ nhớ: $O(N \cdot M)$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.